Architecture Scoring Tool

Sprint 4 Demo

1. Project Context and Requirements

Problem Statement

- **Challenge**: Attribute-Driven Design (ADD) is a powerful approach for architecture design, but there are no tools that help architects follow this methodology effectively
- **Gap**: Architects need a way to evaluate architecture quality, track changes over time, and make data-driven decisions, without looking into long unreadable spreadsheets

Solution Vision

A VS Code extension that enables:

- **Architecture Evaluation**: Score architecture components against attributes
- **Version Tracking**: Compare architecture evolution across Git commits
- **Quality Metrics**: Calculate and visualize quality changes over time
- **Workflow Integration**: Seamless integration with existing development tools

Functional Requirements

- Parse PlantUML component diagrams (text-based, version-controllable)
- Display evolution matrix (components × attributes)
- Support configurable attributes and scoring formulas
- Compare versions via Git history
- Calculate and visualize quality change deltas

Non-Functional Requirements

- **Performance**: Activation < 500ms, parsing < 1s for 50 components
- **Reliability**: 100% data persistence, graceful error handling
- **Compatibility**: Cross-platform (Windows, macOS, Linux)

2. MVP Scope

Core Features for MVP (1/2)

1. **Excalidraw Editor Integration**
    - Custom editor provider for `.addt` files
    - Visual diagram creation and editing
    - VS Code integration
2. **PlantUML Parser**
    - Parse PlantUML text format
    - Extract components, interfaces, relationships
    - Handle syntax errors gracefully

Core Features for MVP (2/2)

3. **Evolution Matrix UI**
    - Interactive table view (components × attributes)
    - Selection for impact values (-1, 0, +1)
    - Real-time updates on diagram changes
4. **Version Comparison (Basic)**
    - Git integration for version history
    - Component-level diff visualization

3. Planning Approach

Strategic Plan (8-Week Roadmap)

Week 1-2: Research & Architecture

- Team formation and research
- Architecture decisions and design
- Technology stack selection

Week 3-4: UI Foundation

- Matrix UI implementation
- File watcher integration

Week 5: Core Parsing

- PlantUML parser implementation
- Itermediate data layer

Strategic Plan (Continued)

Week 6: Diff & Collect all together

- Diff visualizations
- Complete integration

Week 7: Feedback & Polish & Documentation

- Reporting features
- Accessibility improvements

Week 8: Finalization

- Review
- Deployment

Progress Monitoring

- **Weekly sprint reviews** with production-ready increments
- **Performance dashboard** tracking velocity and test coverage
- **Stakeholder meetings** for alignment and feedback
- **Risk management** with identified contingencies

4. Quality Approach

Quality Attribute Scenarios (QAS)

High Priority Requirements:

- **QAS01**: Extension activation < 500ms
- **QAS02**: PlantUML parsing < 1s for 50 components
- **QAS03**: 100% component extraction accuracy
- **QAS04**: 100% data persistence and recovery

Testing Strategy

Unit Testing:

- **Framework**: Jest with TypeScript support
- **Current Coverage**: PlantUML parser

Integration Testing:

- VS Code Extension Test Runner

Code Quality

- **ESLint + Prettier** for code consistency
- **TypeScript strict mode** for type safety
- **Modular architecture** for maintainability

Technology Stack

- React 19 (component-based UI)
- VSCode
- Excalidraw
- Git

6. Progress and Demo

Current Implementation Status

✅ Completed Features

1. **Excalidraw Editor**
    - Custom editor provider registered
    - Full Excalidraw integration in VS Code
    - File format support (`.addt`)
2. **Table view**
    - Table editor for matrix view

Completed Features (Continued)

3. **Modal System**
    - Modal components infrastructure
    - Integration with React webview
    - Custom file format support
4. **PlantUML Parser (In Progress)**
    - Basic parsing structure
    - Component extraction logic

🚧 In Progress

PlantUML → Intermediate → Excalidraw

- **Current Work**: Building the transformation pipeline
- **PlantUML Parser**: Extracting components, relationships, and structure
- **Intermediate Format**: Creating a normalized representation
- **Excalidraw Renderer**: Converting intermediate format to Excalidraw JSON

Demo Walkthrough

Questions & Discussion